Estrutura de Dados II

Prof. Vanessa Matias Leite

1

2° Bimestre

DATA	CONTEÚDO
02/04	Introdução aos Grafos
09/04	---
16/04	Busca em Grafos
23/04	Caminhos mínimos
30/04	Árvore Geradora Mínima Compressão de dados
07/05	Prova 2
14/05	Vista de Prova + Trabalhos
21/05	Vista de Prova + Trabalhos
28/05	Exame

2

Caminhos Mínimos

Dijkstra

3

Grafos

- Vimos que a busca em largura nos dá o menor caminho entre um vértice inicial e todos os demais no grafo.
- O caminho é medido em número de arestas, **ignorando quaisquer pesos que estas tenham.**
- O peso nas arestas crucial, como quando queremos achar a rota mais curta entre duas cidades.

4

Dijkstra

- O que fazer nesses casos?
- Usamos o algoritmo de Dijkstra
- Algoritmo que calcula o caminho mais curto, em termos do **peso total das arestas**, entre um nó inicial e todos os demais nós no grafo.
- O peso total das arestas é a soma dos pesos das arestas que compõem o caminho.

5

Dijkstra

- Para cada vértice v do grafo, mantemos um atributo *d[v]* que é um **limite superior** para o peso do caminho mais curto do nó inicial s a v.
- Dizemos que *d[v]* é uma estimativa de caminho mais curto, inicialmente feito ∞.
- Também armazenamos o vértice que precede v (p[v]- precedente de v) no caminho mais curto de s a v.

6

Dijkstra

- Faça a estimativa de distância de s a qualquer vértice ser infinita
  - Exceto, claro, a distância de s a s, que é 0
  - Ou seja, d[s] = 0 e d[v] = ∞ para todo v + s
- Faça os precedentes dos nós serem um valor qualquer
  - Na prática, podemos fazer p[v] = -1, já que não temos vértice de indice-1 na estrutura GRAFO
- Marco todos os vértices como “abertos”.

7

Dijkstra

Enquanto houver vértice aberto:
- Escolha o vértice aberto u cuja estimativa seja a menor dentre os demais abertos
- Feche u
- Para todo nó aberto v na adjacência de u:
  - Some d[u] ao peso da aresta (u, v)
  - Caso a soma seja menor que d[v], atualize d[v] e faça p[v] = u
  - Procedimento chamado de relaxamento da aresta (u, v)

8

Dijkstra

Inicialize o grafo com d[s] = 0, d [v] = ∞, para todo v ≠ s, e p[v] = -1 para todo v

Faça aberto[v] = true para todo v no grafo

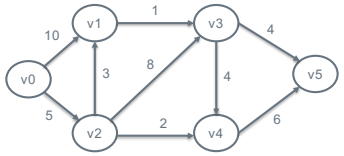
Enquanto houver vértice aberto:
- Escolha u cuja estimativa seja a menor dentre os abertos
- Feche u
- Para todo nó aberto v na adjacência de u:
  - Relaxe a aresta (u, v)

9

Dijkstra

Faça as inicializações;

Enquanto houver vértice aberto:
- Escolha u cuja estimativa seja a menor dentre os abertos
- Feche u
- Para todo nó aberto v na adjacência de u:
  - Relaxe a aresta (u, v)



10

Dijkstra – Observação

- Fechamos um nó u somente se já conhecemos a menor distância deste ao nó inicial s
- Então fazemos o relaxamento de seus vizinhos:
  - Sabemos a menor distância de s a u
  - Temos a estimativa de s a v, v vizinho de u
  - Buscamos saber se a distância de s a u mais a distância (u, v) é melhor que a estimativa atual de s a v. Se for, atualizamos essa estimativa.

11

Dijkstra – Observação

- Note que, além da distância, o caminho também pode ser obtido!
- Basta olhar os antecedentes, percorrendo do nó final ao inicial.

12

Dijkstra

```c
typedef struct adjacencia {
    int vertice;
    TIPOPESO peso;
    struct adjacencia *prox;
} ADJACENCIA;


typedef struct vertice {
    /* Dados armazenados vão aqui */
    ADJACENCIA *cab;

} VERTICE;

typedef struct grafo {
    int vertices;
    int arestas;
    VERTICE *adj;

} GRAFO;
```

13

Dijkstra

- Começamos inicializando o grafo
- Fazemos d[v] = ∞ para todo nó que não o inicial s
- INT_MAX está na biblioteca limits.h

```c
void inicializaD (GRAFO *g, int *d, int *p, int s) {
    int v;
    for (v=0;v<g->vertices;v++) {
        d[v] = INT_MAX/2;
        p[v] = -1;

    }
    d[s] = 0;
}
```

14

Dijkstra

- Identificamos a aresta (u,v)
- Relaxamos (u,v), atualizando a estimativa e o predecessor.

```
void relaxa (GRAFO *g, int *d, int*p, int u, int v){
    ADJACENCIA *ad = g->adj [u] .cab;
    while (ad && ad->vertice != v)
        ad = ad->prox;
    if (ad) {
        if (d[v] > d[u] + ad->peso) {
            d[v] = d [u] + ad->peso;
            p[v] = u;

        }

    }
}
```

15

Dijkstra

```
int *dijkstra(GRAFO *g,int s) {
    int * = (int *) malloc(g->vertices*sizeof(int));
    int p[g->vertices];
    bool aberto [g->vertices];
    InicializaD (g,d,p,s);

    int i;
    for (i=0; i<g->vertices; i++)
        aberto [i] = true;

    while (existeAberto (g, aberto)) {
        int u = menorDist (g, aberto,d) ;
        aberto [u] = false;
        ADJACENCIA *ad = g->adj [u].cab;
        while (ad) {
            relaxa (g,d,p,u,ad->vertice);
            ad = ad->prox;

        }

    }

    return (d);
}
```

16

Dijkstra

- existeAberto simplesmente varre todos os vértices do grafo, retornando *true* se encontrar algum aberto, e *false* se não encontrar

```
bool existeAberto (GRAFO *g, int *aberto) {
    int i;
    for (i=0;1<g->vertices;i++)
        if (aberto [i])  return (true);
    return (false);
}
```

17

Dijkstra

- menorDist busca o primeiro nó aberto. Assume então esse nó como o de menor estimativa de distância.

```
int menorDist (GRAFO *g,int *aberto, int *d) {
    int i;

    for (i=0; i<g->vertices; i++)
        if (aberto [i]) break;

    if (i==g->vertices) return (-1);
    int menor = i;
    for (i-menor+1; i<g->vertices; i++) {
        if (aberto [i] && (d[menor]>d[i]))
        menor = i;
    }
    return (menor);
}
```

18

Dijkstra

Dijkstra tem um limitante importante:

- Os pesos das arestas devem ser não-negativos
- Além disso, a implementação de existeAberto e menorDist é crucial para a velocidade do algoritmo A implementação aqui mostrada não é a melhor. Para uma melhor, busque uma implementação com filas de prioridade

19

Caminhos mínimos

Bellman-Ford

20

Bellman-Ford

- O algoritmo de Bellman-Ford
- permite a existência de arestas de peso negativo e produz a resposta correta.
- informa se grafo contém ciclo de peso negativo acessível da origem
- não entra em *loop*

21

Bellman-Ford

Estruturas utilizadas:

- Vetores p e d:
- p[v]: pai do vértice v
- d[u] : distância da origem s até u

22

Bellman-Ford

Relaxamento:

- ▷ Também utilizado pelo algoritmo de Dijkstra
- ▷ Consiste em verificar se pode ser encontrado um caminho mais curto para v (do que aquele encontrado até o momento), passando pelo vértice corrente u.

23

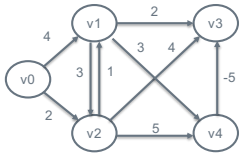
Bellman-Ford

```
BELLMAN-FORD (G(V,E),s)
    INICIALIZA (G,S)
    for i= 1 to n-1
        for each (u,v) ∈ E
            RELAXA (u,v)
        end-for
    end-for
    for each (u,v) ∈ E
      if d[v]>d[u] + w(u,v)
        return false
      end-if
    end-for
      return true
```

Relaxamento das arestas

Analisa ciclos com pesos negativos

▷



24